

# AITA-INTELLIGENT — Software Requirements Specification (SRS)

**Version:** 1.2

**Date Created:** 09/07/2026

**Development Team:** SWP391 – Group 3

**Semester:** Fall 2026 – FPT University

## Table of Contents

1. [Introduction](#)
2. [System Overview](#)
3. [System Actors](#)
4. [User Stories](#)
5. [Detailed Use Case Specifications](#)
6. [UML Diagrams](#)
7. [Non-Functional Requirements](#)
8. [System Constraints](#)

## 1. Introduction

### 1.1. Document Purpose

This Software Requirements Specification (SRS) document describes all functional and non-functional requirements of the **AITA-Intelligent** system (AI-Powered Teaching Assistant & AST Code Analytics Platform). This document serves as the foundation for system design, development, testing, and acceptance within the scope of the SWP391 – Software Development Project course.

### 1.2. Project Scope

AITA-Intelligent is an intelligent programming education support platform, consisting of:

- **Web Portal (PWA-ready)** for students and lecturers (Mobile Native is Future Scope).
- **Automated code grading system** via Judge0 API sandbox (Code Execution Engine).
- **Generative AI module** (GenAI) supporting assignment creation, Clean Code grading, and compilation error explanation.
- **Source code plagiarism detection tool** based on Abstract Syntax Tree (AST) analysis combined with the Winnowing algorithm.
- **Background processing queue** (Redis Queue) and team contribution analysis via Git Analytics.

### 1.3. Intended Audience

| Audience                    | Purpose                           |
|-----------------------------|-----------------------------------|
| Instructor                  | Product evaluation and acceptance |
| Development Team (Dev Team) | Design and development reference  |
| P2P Peer Reviewers          | Requirement completeness review   |
| Defense Committee           | Final evaluation                  |

### 1.4. Glossary

| Term               | Definition                                                                                    |
|--------------------|-----------------------------------------------------------------------------------------------|
| <b>AST</b>         | Abstract Syntax Tree – A tree representation of the logical structure of source code          |
| <b>Winnowing</b>   | An algorithm for creating digital fingerprints from k-gram sequences for text/code matching   |
| <b>Sandbox</b>     | An isolated code execution environment (provided by Judge0/Piston API) with limited resources |
| <b>JWT</b>         | JSON Web Token – A stateless authentication standard                                          |
| <b>SSO</b>         | Single Sign-On – One-click login via Google OAuth 2.0                                         |
| <b>BullMQ</b>      | A Redis-based queue library for Node.js                                                       |
| <b>LLM</b>         | Large Language Model – e.g., GPT-4o, Gemini                                                   |
| <b>ERD</b>         | Entity-Relationship Diagram                                                                   |
| <b>SRS</b>         | Software Requirements Specification                                                           |
| <b>P2P</b>         | Peer-to-Peer – Cross-team evaluation                                                          |
| <b>UAT</b>         | User Acceptance Testing                                                                       |
| <b>CI/CD</b>       | Continuous Integration / Continuous Deployment                                                |
| <b>LOC</b>         | Lines of Code                                                                                 |
| <b>k-gram</b>      | A contiguous subsequence of k elements, used in the Winnowing algorithm                       |
| <b>Fingerprint</b> | Digital fingerprint – A set of hash values representing a document/source code                |

### 1.5. References

- Schleimer, S., Wilkerson, D.S., Aiken, A. (2003). *"Winnowing: Local Algorithms for Document Fingerprinting"*. ACM SIGMOD 2003.
- Parr, T. (2010). *"Language Implementation Patterns"*. Pragmatic Bookshelf.
- Sommerville, I. (2016). *"Software Engineering"*, 10th Edition. Pearson.
- Judge0 API Documentation - <https://ce.judge0.com/>
- OpenAI API Reference – <https://platform.openai.com/docs/api-reference>

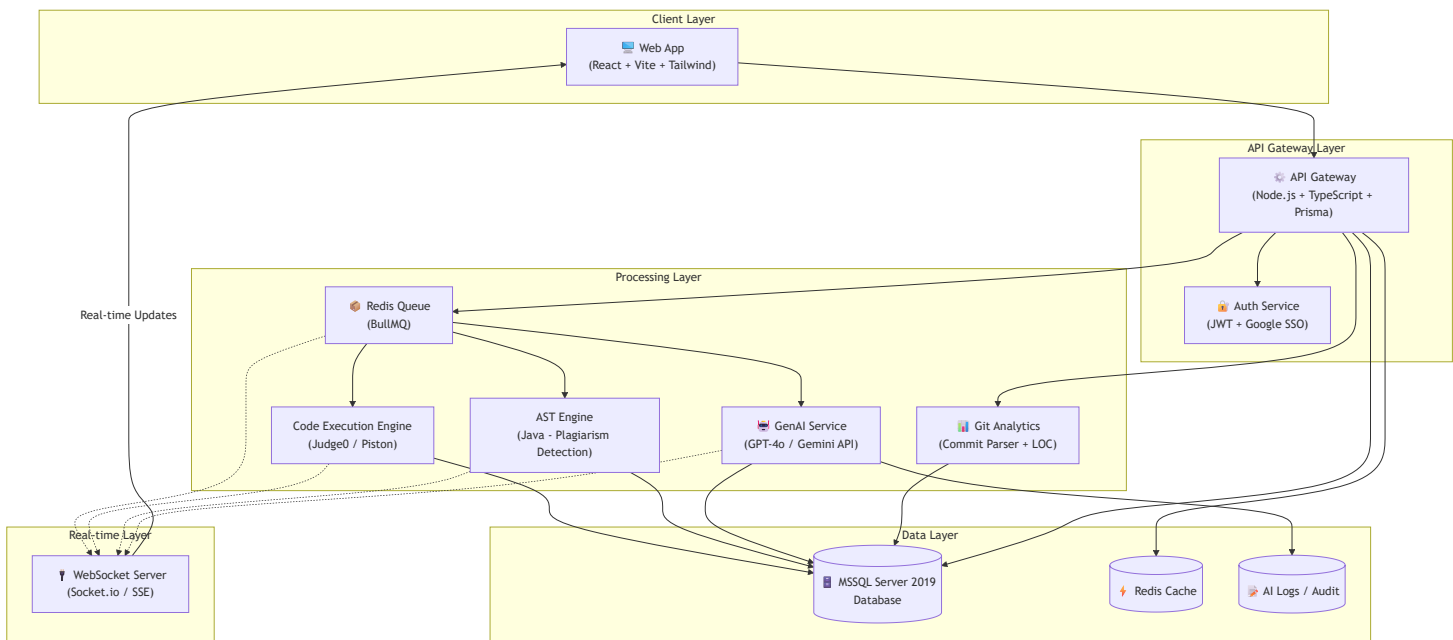
## 2. System Overview

### 2.1. Product Context

The AITA-Intelligent system addresses real-world challenges in programming education at universities:

- **Time-consuming manual grading:** Lecturers spend hours grading code for large classes. AITA automates this process using Code Execution Engine.
- **Sophisticated source code fraud:** Students use AI (ChatGPT, Copilot) to write submissions and then disguise them (rename variables, reorder functions). Traditional text comparison tools are ineffective. AITA uses AST + Winking for detection.
- **Lack of quality feedback:** Students only see scores without understanding their mistakes. AITA integrates LLM to explain errors in natural language.

### 2.2. High-Level Architecture

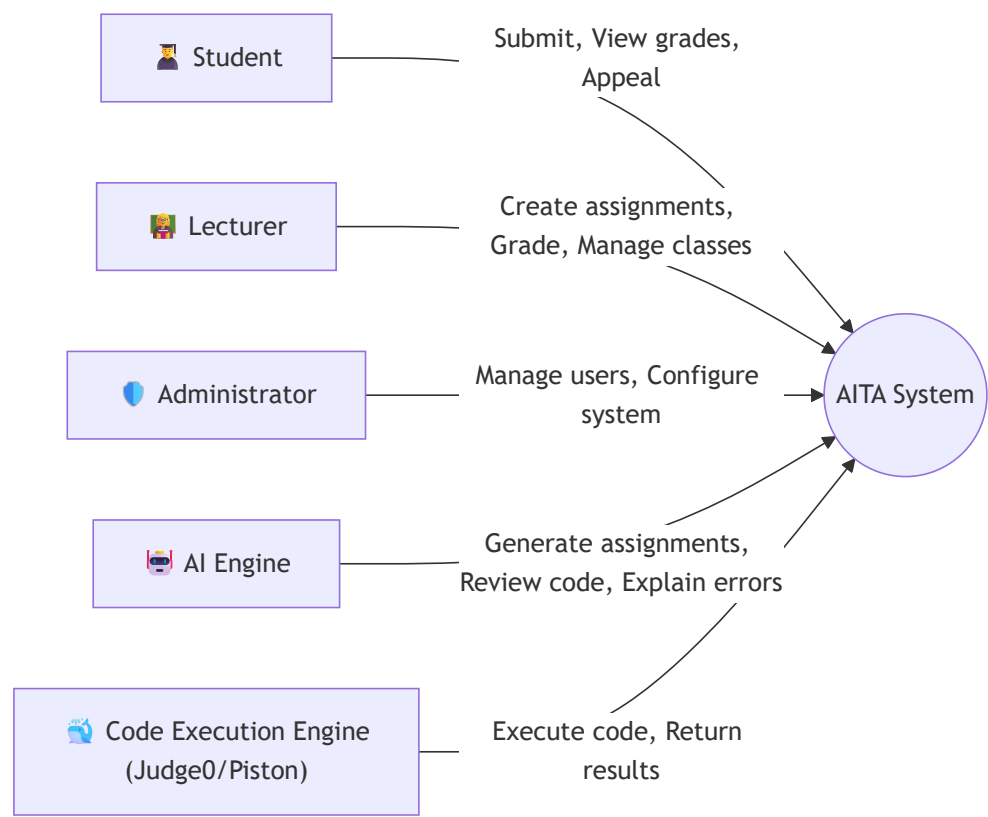


### 2.3. Core Subsystems

| # | Subsystem                   | Description                                                  | Key Technology                               |
|---|-----------------------------|--------------------------------------------------------------|----------------------------------------------|
| 1 | Portal & Auth               | User interface, authentication, authorization                | React, Vite, Tailwind, JWT, Google OAuth 2.0 |
| 2 | Code Execution Engine       | Automated code grading in isolated sandbox                   | Judge0 API, Node.js                          |
| 3 | GenAI Core & Review Hub     | Assignment generation, Clean Code grading, error explanation | OpenAI GPT-4o / Gemini API, LangChain        |
| 4 | AST Plagiarism Detection    | Source code plagiarism detection using AST + Winking         | ANTLR4 (Java grammar), FastAPI wrapper       |
| 5 | Redis Queue & Git Analytics | Background processing queue, team contribution analysis      | BullMQ, Redis, Git CLI Parser                |

# 3. System Actors

## 3.1. Actor Diagram



## 3.2. Detailed Actor Descriptions

| Actor         | Role                                                                                                              | Key Permissions                                                                                                                                                                                      |
|---------------|-------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Student       | Students use the system to submit assignments, view grading results, and appeal if they disagree with the outcome | SSO login (@fpt.edu.vn), view assignment list, upload .zip file, view real-time results, submit appeal                                                                                               |
| Lecturer      | Lecturers create classes, design exams, manage grading rubrics, monitor results, and handle complaints            | Create/edit/delete classes, import student lists from Excel, create assignments + test cases, request AI-generated assignments/rubrics, view statistical reports, handle appeals, view Git Analytics |
| Administrator | System administrators manage accounts, configure Sandbox settings, and monitor system operations                  | Manage users, configure Sandbox (RAM, CPU limits), manage API keys (OpenAI/Gemini), view audit logs                                                                                                  |
| AI Engine     | Automated system actor – Not a human. Performs content generation and code quality evaluation tasks               | Generate assignments from lecturer descriptions, generate rubrics, evaluate Clean Code (SOLID, naming, architecture), explain compilation errors in natural language                                 |

| Actor                        | Role                                                                                    | Key Permissions                                                                                   |
|------------------------------|-----------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------|
| <b>Code Execution Engine</b> | Automated system actor. An isolated runtime that executes student-submitted source code | Send request to Judge0/Piston API, receive compile/run results, enforce timeout at API call level |

## 4. User Stories

### 4.1. Student Features

| ID           | User Story                                                                                                                                                                                                             | Acceptance Criteria                                                                                                                                                                                                                                                                                                                                         | Priority                                                                                     |
|--------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------|
| <b>US-01</b> | As a <b>student</b> , I want to <b>log in using my FPT Google account</b> ( @fpt.edu.vn / @fe.edu.vn ) so I don't need to create a new account and my identity is verified.                                            | <ul style="list-style-type: none"> <li>- Only accept FPT domain emails</li> <li>- Redirect to Dashboard after login</li> <li>- JWT stored in HttpOnly Cookie</li> </ul>                                                                                                                                                                                     |  High     |
| <b>US-02</b> | As a <b>student</b> , I want to <b>view the list of Assignments</b> for my enrolled class, so I can track deadlines and submission status.                                                                             | <ul style="list-style-type: none"> <li>- Display assignment name, deadline, status (Not submitted / Submitted / Graded)</li> <li>- Sort by nearest deadline</li> </ul>                                                                                                                                                                                      |  High     |
| <b>US-03</b> | As a <b>student</b> , I want to <b>submit assignments as a .zip file</b> containing source code, so the system can automatically grade them for me.                                                                    | <ul style="list-style-type: none"> <li>- Only accept .zip files (max 10MB)</li> <li>- File hashed with SHA-256 for integrity verification</li> <li>- System displays "Submission received" immediately</li> <li>- Reject submission with error message if student has already submitted <math>\geq 5</math> times for this assignment (see R-01)</li> </ul> |  High   |
| <b>US-04</b> | As a <b>student</b> , I want to <b>view grading results in real-time</b> (test case scores, Clean Code score, plagiarism check results) as soon as the system finishes processing, without needing to reload the page. | <ul style="list-style-type: none"> <li>- Real-time updates via WebSocket</li> <li>- Clearly display: Passed/Failed for each test case</li> <li>- Display Clean Code score with AI explanation</li> <li>- Display plagiarism similarity percentage (0-30% Safe, 30-60% Warning, &gt;60% Danger/Flag)</li> </ul>                                              |  High   |
| <b>US-05</b> | As a <b>student</b> , I want to <b>submit an Appeal</b> if I believe the automated grading result is inaccurate, so the lecturer can re-evaluate.                                                                      | <ul style="list-style-type: none"> <li>- Appeal form with "Reason" field (textarea)</li> <li>- Appeal status: Pending / Accepted / Rejected</li> <li>- Student receives notification when lecturer responds</li> </ul>                                                                                                                                      |  Medium |

## 4.2. Lecturer Features

| ID    | User Story                                                                                                                                                                                             | Acceptance Criteria                                                                                                                                                                                                                                                                                                                                  | Priority                                                                                     |
|-------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------|
| US-06 | As a <b>lecturer</b> , I want to <b>create a new class and import student lists from an Excel file</b> , to save time on manual data entry.                                                            | <ul style="list-style-type: none"> <li>- Upload .xlsx file, system auto-parses columns: Student ID, Full Name, Email</li> <li>- Use DB Transaction to ensure full import or rollback on error</li> <li>- Report successful/failed import counts</li> </ul>                                                                                           |  High     |
| US-07 | As a <b>lecturer</b> , I want to <b>create Assignments with test cases</b> (StdIn/StdOut format), so the system can automatically grade student submissions.                                           | <ul style="list-style-type: none"> <li>- Input title, description, deadline</li> <li>- Add multiple test cases (each with: Input, Expected Output, score weight)</li> <li>- Select programming language for Sandbox (Java, Python, C#)</li> </ul>                                                                                                    |  High     |
| US-08 | <i>(Simplified – MVP)</i> As a <b>lecturer</b> , I want to <b>request AI to generate a basic assignment draft</b> from my brief description, to reduce assignment preparation time.                    | <ul style="list-style-type: none"> <li>- Lecturer enters a prompt describing the topic (e.g., "Assignment on linked list")</li> <li>- AI returns: basic assignment description and sample test cases</li> <li>- Lecturer must review and finalize before saving</li> <li>- <i>MVP Note: Full rubric auto-generation is a stretch goal</i></li> </ul> |  Medium   |
| US-09 | As a <b>lecturer</b> , I want to <b>view a Dashboard with overview statistics</b> (submission rate, score distribution, plagiarism suspects) for each assignment, to quickly assess class performance. | <ul style="list-style-type: none"> <li>- Pie chart: submission rate</li> <li>- Bar chart: score distribution</li> <li>- Table: Top 10 submission pairs with highest similarity % (AST Plagiarism)</li> </ul>                                                                                                                                         |  High   |
| US-10 | As a <b>lecturer</b> , I want to <b>handle student appeals</b> (Accept or Reject with reason), to ensure fairness in grading.                                                                          | <ul style="list-style-type: none"> <li>- List of Pending appeals</li> <li>- Review code + original grading results</li> <li>- Accept (re-grade) or Reject (with reason) button</li> </ul>                                                                                                                                                            |  Medium |
| US-11 | <i>(Simplified – MVP)</i> As a <b>lecturer</b> , I want to <b>view basic Git Analytics</b> for each team member (commits, LOC), to get a high-level view of individual contributions.                  | <ul style="list-style-type: none"> <li>- Input GitHub Repo URL of the team</li> <li>- Display: Total commits, LOC added/removed per member</li> <li>- <i>MVP Note: PR analysis and contribution timeline charts are planned for future iterations</i></li> </ul>                                                                                     |  Medium |

## 4.3. System / AI / Code Execution Features

| ID    | User Story                                                                                                                                                                                                                                                                                                                                 | Acceptance Criteria                                                                                                                                                                                                                                                                                                                                                                                                                  | Priority                                                                                     |
|-------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------|
| US-12 | As the <b>system</b> , when receiving a .zip submission file, I must <b>enqueue the submission into Redis (BullMQ)</b> for asynchronous processing, to avoid blocking the main API thread.                                                                                                                                                 | <ul style="list-style-type: none"> <li>- Submission enqueued with metadata: submissionId, studentId, language</li> <li>- Queue has retry mechanism (max 3 retries) on job failure</li> <li>- API immediately returns status 202 Accepted</li> </ul>                                                                                                                                                                                  |  High     |
| US-13 | As the <b>Code Execution Engine</b> , when receiving a job from Redis Queue, I must <b>execute student code in an isolated sandbox</b> (using Judge0/Piston) with limits: 512MB RAM, limited CPU, <b>disabled network access</b> , run with test cases, then return results.                                                               | <ul style="list-style-type: none"> <li>- Sandbox auto-cleans after completion or timeout (30 seconds)</li> <li>- Judge0/Piston handles: process isolation, resource limiting, security</li> <li>- Return results: Passed/Failed for each test case + stdout/stderr</li> </ul>                                                                                                                                                        |  High     |
| US-14 | As the <b>AST Engine system</b> , after code execution finishes, I must <b>convert source code to an AST, remove surface-level disguises</b> (variable names, comments, function order), <b>apply Winnowing</b> to create fingerprints, and <b>compare against all other submissions</b> in the same assignment to calculate similarity %. | <ul style="list-style-type: none"> <li>- <b>Primary language: Java</b> (ANTLR parser)</li> <li>- <i>Python/C# support planned for future iterations</i></li> <li>- Remove: variable names, function names, comments, whitespace, function order</li> <li>- Winnowing: k-gram size = 25, window size = 40</li> <li>- Store fingerprints in ASTFingerprints table</li> <li>- Create Similarity Matrix: all submission pairs</li> </ul> |  High   |
| US-15 | As the <b>AI system (LLM)</b> , after the Code Execution Engine finishes grading, I must <b>evaluate source code quality</b> (Clean Code, SOLID, naming convention, layer architecture) and <b>explain compilation errors in Vietnamese natural language</b> so students can understand.                                                   | <ul style="list-style-type: none"> <li>- Use GPT-4o or Gemini API (exponential backoff retry)</li> <li>- Chain-of-Thought + Few-Shot Learning prompts</li> <li>- Return: Clean Code score (0-100), specific comments per file, compilation error explanation if any</li> <li>- Timeout: max 60 seconds/request</li> </ul>                                                                                                            |  Medium |

# 5. Detailed Use Case Specifications

## UC-01: Student Submits Assignment (Submit Assignment)

| Attribute        | Description                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  |
|------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Primary Actor    | Student                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                      |
| Secondary Actors | Code Execution Engine, AST Engine, AI Engine                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                 |
| Preconditions    | Student is logged in, Assignment deadline has not passed, Student has submitted fewer than 5 times for this assignment                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                       |
| Main Flow        | <div>1. Student selects Assignment from the list</div> <div>2. Student uploads <code>.zip</code> file containing source code</div> <div>3. System validates file (checks size <math>\leq 10\text{MB}</math>, <code>.zip</code> format)</div> <div>4. System hashes file with SHA-256 for integrity verification</div> <div>5. System creates <code>Submission</code> record with <code>PENDING</code> status</div> <div>6. System pushes job to Redis Queue (BullMQ)</div> <div>7. Redis Queue dispatches job to Worker</div> <div>8. Code Execution Engine sends submission to Judge0 API, receives test case results</div> <div>9. System updates status to <code>GRADING</code></div> <div>10. AST Engine parses source code, creates fingerprint, checks for plagiarism</div> <div>11. AI Engine evaluates Clean Code + explains errors</div> <div>12. System aggregates final score; updates status to <code>COMPLETED</code> if similarity <math>\leq 60\%</math>, or <code>SCORE_WITHHELD</code> if similarity <math>&gt; 60\%</math> (pending lecturer review)</div> <div>13. Sends real-time results to Student via WebSocket</div> |
| Alternative Flow | <div><b>4a.</b> File is corrupt or exceeds size limit <math>\rightarrow</math> Display error, request resubmission</div> <div><b>8a.</b> Student code contains malware (fork bomb, <code>/proc</code> access, disk abuse) <math>\rightarrow</math> Judge0 detects malicious signal <math>\rightarrow</math> Status <code>SECURITY_VIOLATION</code></div> <div><b>8b.</b> Code fails to compile <math>\rightarrow</math> Return compilation error + AI error explanation</div> <div><b>8c.</b> Code execution exceeds 30 seconds without malicious signal <math>\rightarrow</math> Status <code>TIMEOUT</code>, notify student to optimize algorithm</div> <div><b>10a.</b> Similarity detected <math>&gt; 60\%</math> <math>\rightarrow</math> Flag as <code>PLAGIARISM_DETECTED</code>, notify lecturer</div>                                                                                                                                                                                                                                                                                                                               |
| Exception Flow   | <div><b>7a.</b> Redis Queue is full or down <math>\rightarrow</math> System retries 3 times, if still failing <math>\rightarrow</math> Status <code>QUEUE_ERROR</code>, notify Admin</div> <div><b>11a.</b> OpenAI/Gemini API timeout <math>\rightarrow</math> Skip Clean Code score, grade based on test cases + AST, mark "AI Review Pending"</div>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                        |
| Postconditions   | <code>Submission</code> record is updated with final status and score. Student receives results on the interface.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                            |

## UC-02: Lecturer Creates Assignment with AI (Create Assignment with AI)

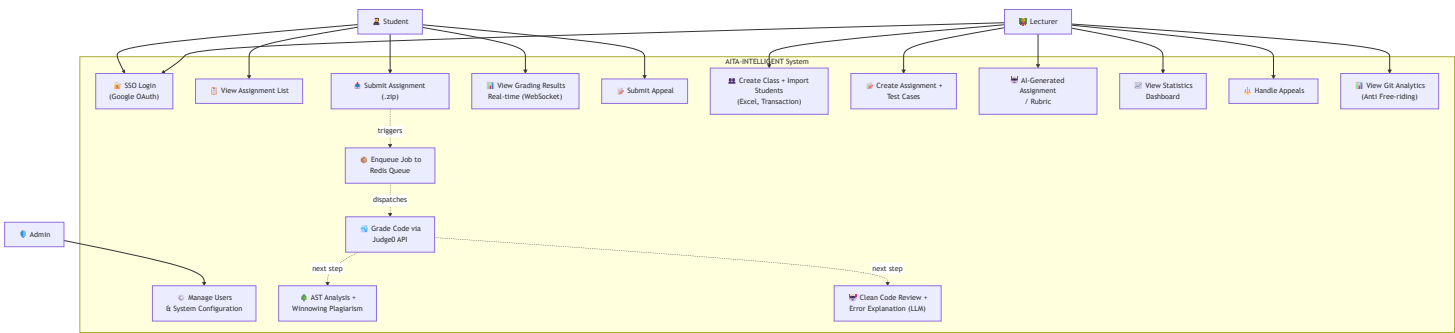
| Attribute        | Description                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                      |
|------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Primary Actor    | Lecturer                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                         |
| Secondary Actors | AI Engine                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                        |
| Preconditions    | Lecturer is logged in, has created at least 1 class                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                              |
| Main Flow        | <ol style="list-style-type: none"><li>1. Lecturer selects class and clicks "Create New Assignment"</li><li>2. Lecturer enters: Title, General Description, Programming Language, Deadline</li><li>3. (Optional) Lecturer clicks "Generate with AI": enters a topic description prompt</li><li>4. AI Engine returns: Detailed assignment, sample Input/Output, Grading Rubric</li><li>5. Lecturer reviews and edits AI-generated content</li><li>6. Lecturer adds Test Cases (StdIn → Expected StdOut) manually or from AI</li><li>7. Lecturer clicks "Save &amp; Publish"</li><li>8. System creates Assignment record, sends notification to all students in the class</li></ol> |
| Alternative Flow | <p><b>3a.</b> Lecturer doesn't use AI → Manually enters the entire assignment</p> <p><b>4a.</b> AI returns unsuitable results → Lecturer clicks "Regenerate" with a different prompt</p> <p><b>6a.</b> Test cases are invalid (Expected Output is empty) → System warns and requests correction</p>                                                                                                                                                                                                                                                                                                                                                                              |
| Postconditions   | Assignment is created and displayed in the class assignment list.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                |

## UC-03: Lecturer Imports Student List (Bulk Import Students)

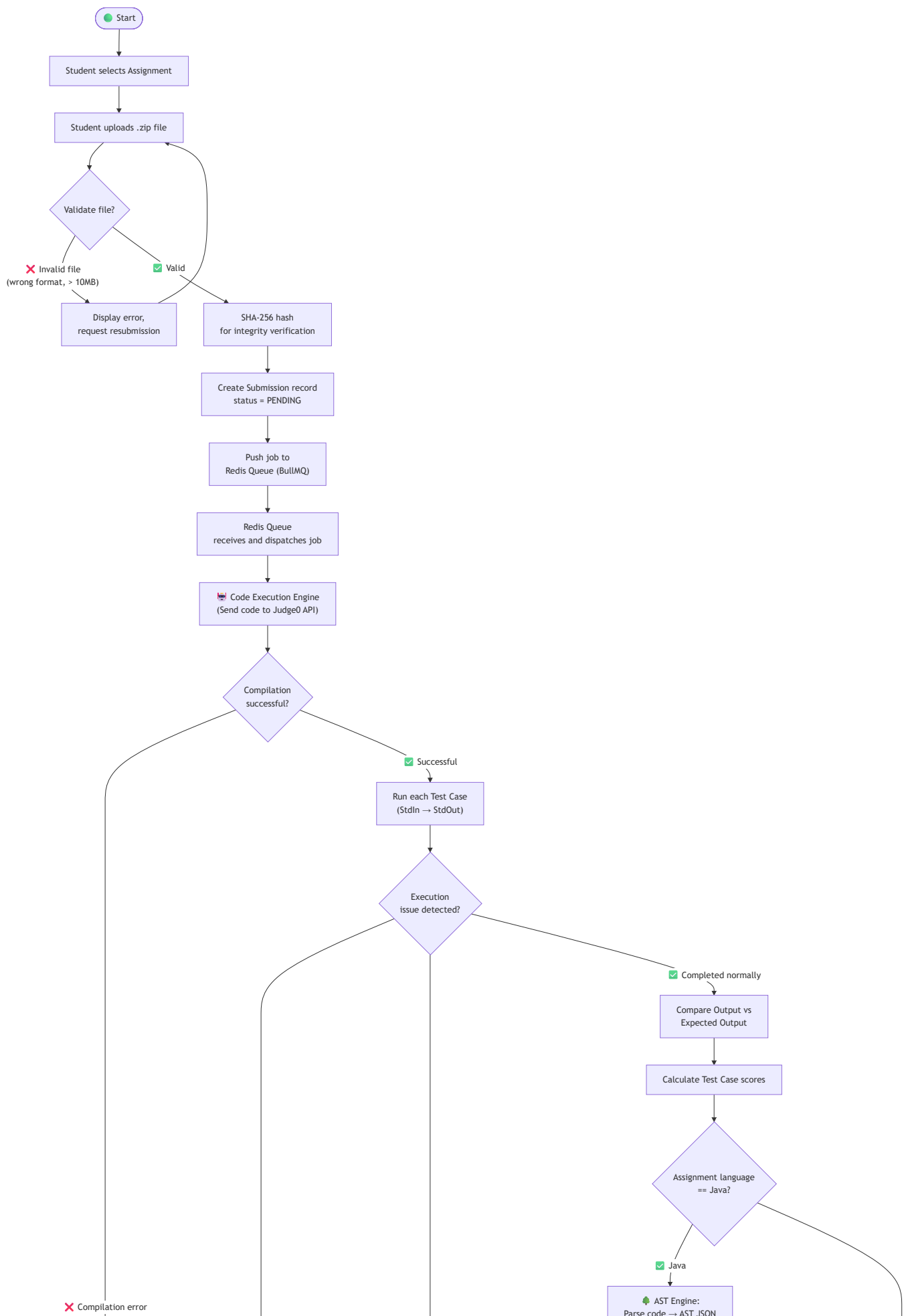
| Attribute        | Description                                                                                                                                                                                                                                                                                                                                                                                                                                                                |
|------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Primary Actor    | Lecturer                                                                                                                                                                                                                                                                                                                                                                                                                                                                   |
| Preconditions    | Lecturer has created a class, has a properly formatted Excel file                                                                                                                                                                                                                                                                                                                                                                                                          |
| Main Flow        | <ol style="list-style-type: none"><li>1. Lecturer selects class, clicks "Import Students"</li><li>2. Lecturer uploads .xlsx file</li><li>3. System parses file, extracts: Student ID, Full Name, Email</li><li>4. System displays preview list (for lecturer to verify)</li><li>5. Lecturer clicks "Confirm Import"</li><li>6. System uses <b>SQL Transaction</b> to insert all students</li><li>7. Displays results: X students successful, Y duplicate records</li></ol> |
| Alternative Flow | <p><b>3a.</b> Excel file has wrong format (missing columns) → Display detailed error, request re-upload</p> <p><b>6a.</b> Transaction fails (e.g., duplicate email) → Rollback entirely, report specific duplicate rows</p>                                                                                                                                                                                                                                                |
| Postconditions   | Student list is added to the class. New accounts are created (if not already existing).                                                                                                                                                                                                                                                                                                                                                                                    |

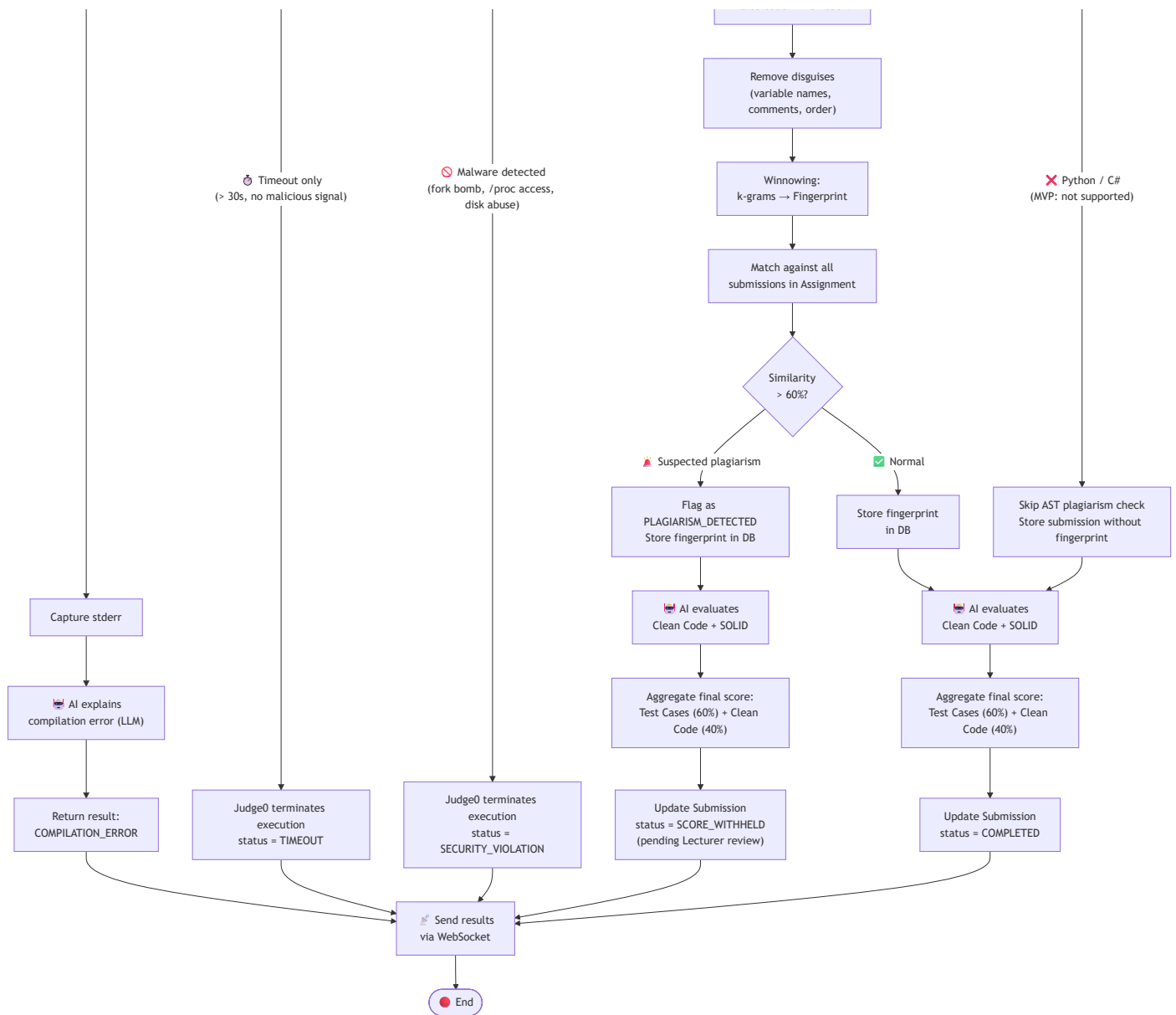
# 6. UML Diagrams

## 6.1. Use Case Diagram (Overview)

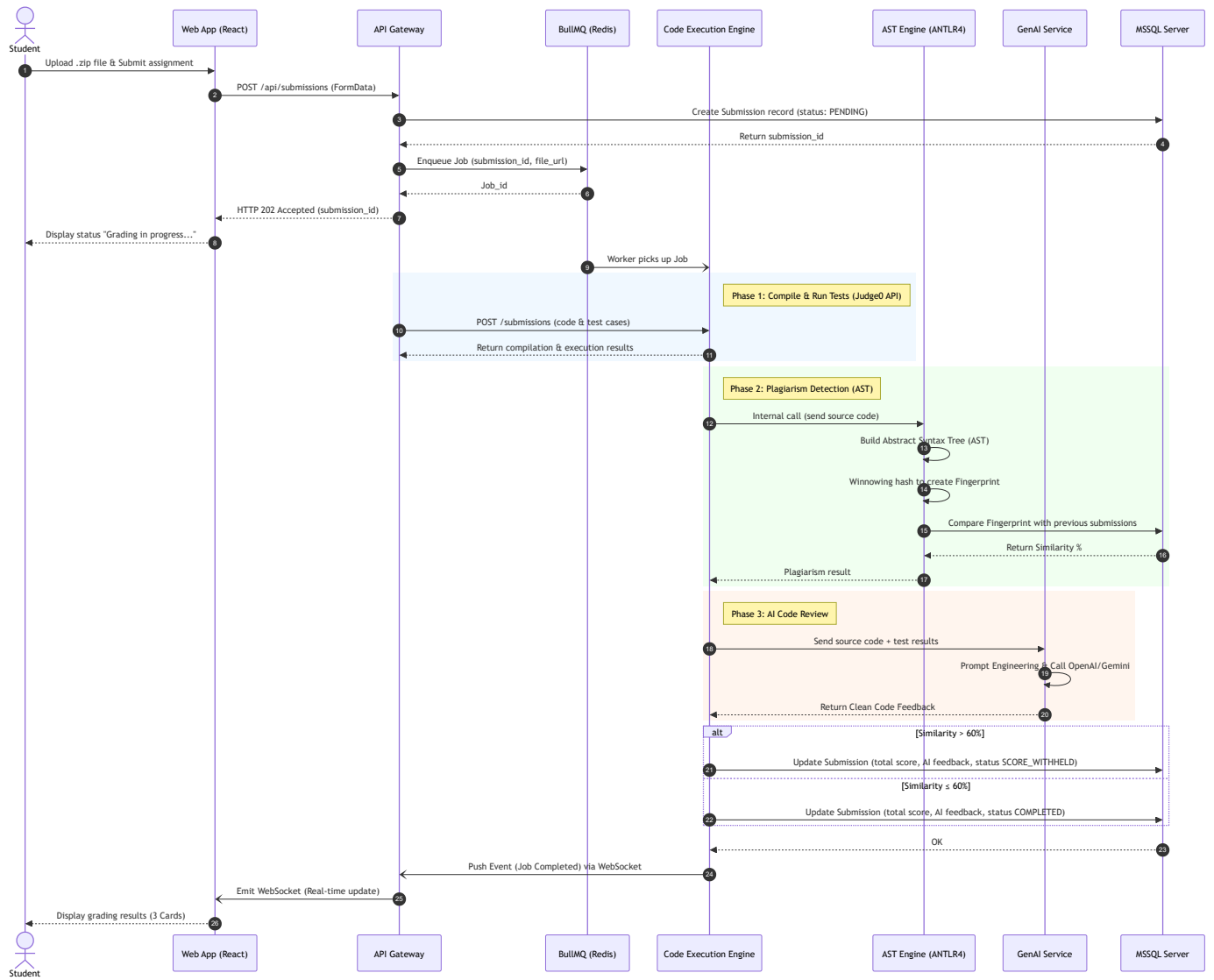


**6.2. Activity Diagram – Submission and Automated Grading Flow**



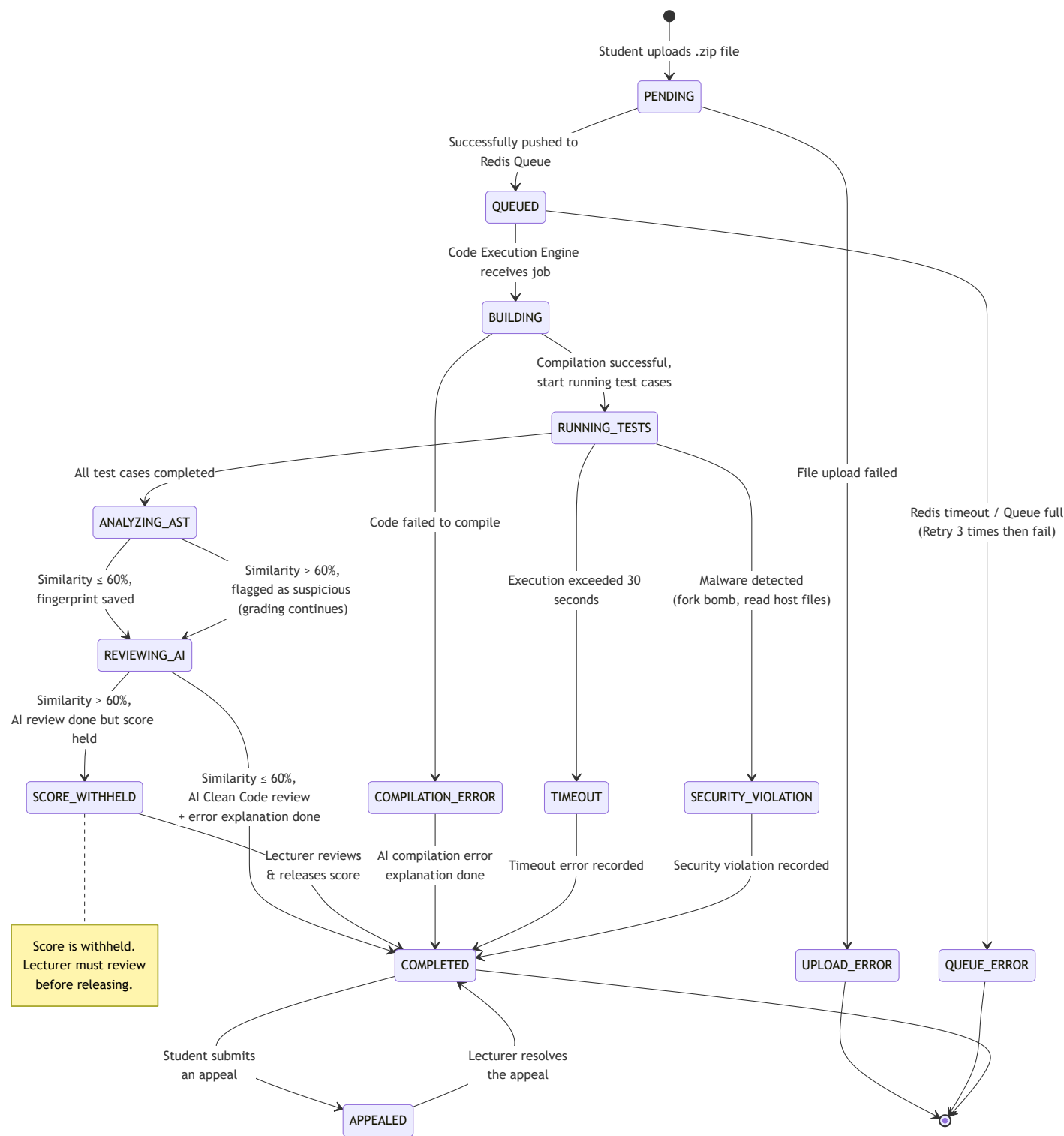


## 6.3. Sequence Diagram – Submission Grading API Flow

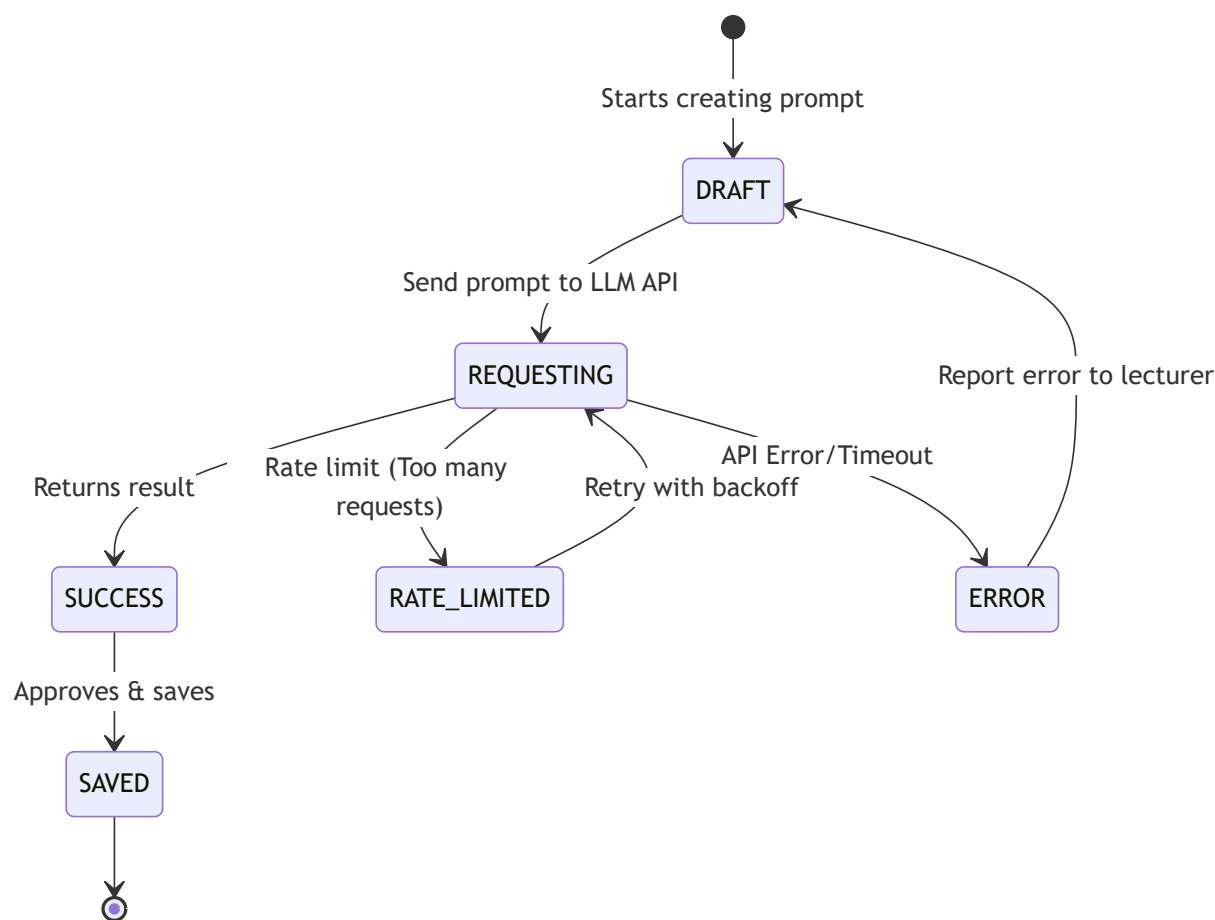


# 6.4. State Diagrams

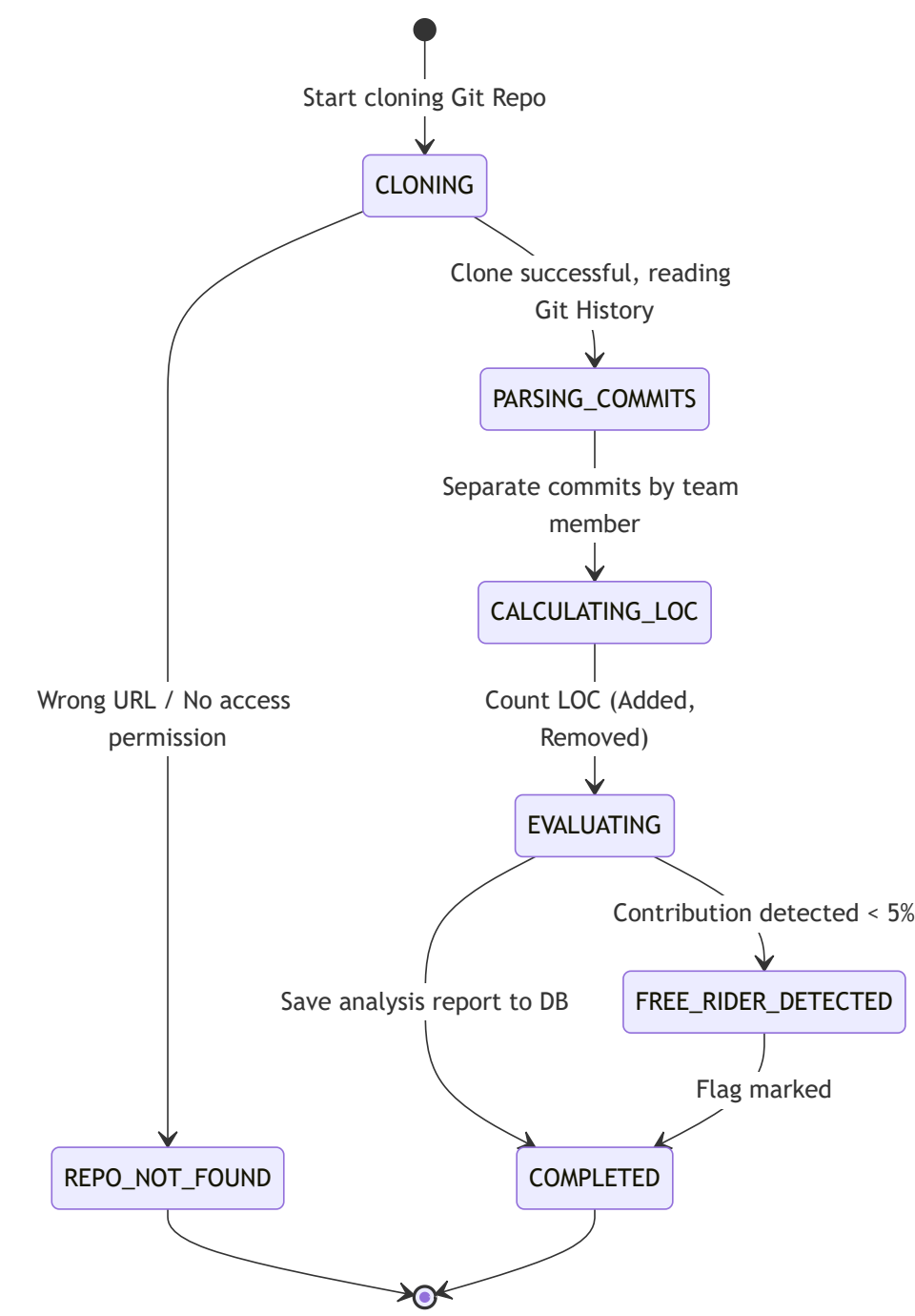
## 6.4.1. Submission Lifecycle



6.4.2. GenAI Request Lifecycle

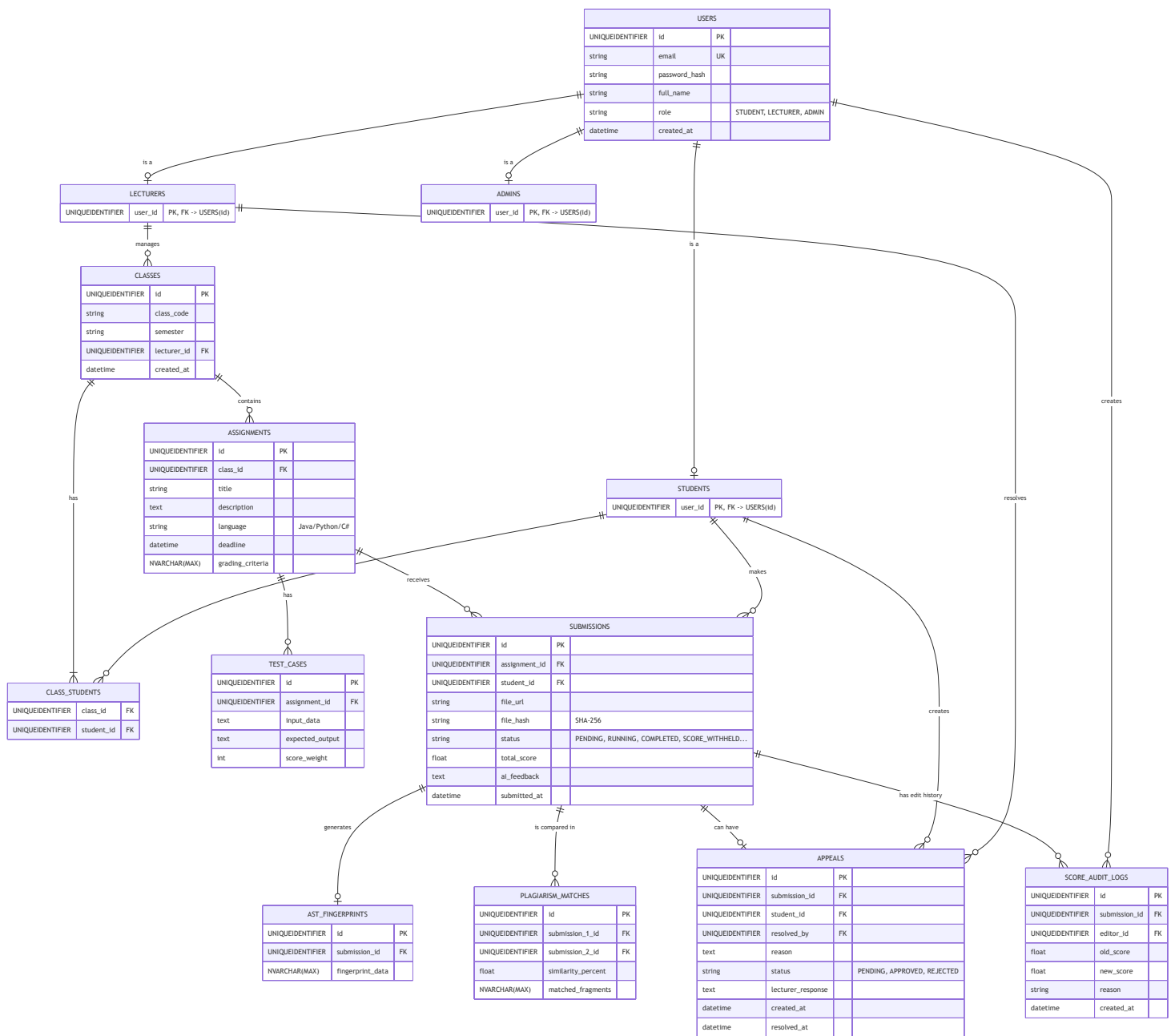


### 6.4.3. Git Analytics Process Lifecycle



### 6.5. ERD (Entity-Relationship Diagram)

**Design Note — Class Table Inheritance (CTI):** The `USERS` table stores only attributes shared across all roles, while role-specific data is separated into child tables ( `STUDENTS` , `LECTURERS` , `ADMINS` ) linked via a 1-to-1 FK on `user_id` . This pattern ensures referential integrity at the database level — for example, `CLASSES.lecturer_id` can only reference a row that actually exists in `LECTURERS` , preventing accidental assignment of a student ID to a lecturer-only column. The `role` discriminator column is retained in `USERS` so the application layer knows which child table to JOIN without a multi-table probe.



## 6.6. Data Dictionary

| Table | Column        | Data Type        | Constraints      | Description                           |
|-------|---------------|------------------|------------------|---------------------------------------|
| USERS | id            | UNIQUEIDENTIFIER | PK               | Primary key, user identifier          |
|       | email         | VARCHAR(255)     | UNIQUE, NOT NULL | FPT email for login                   |
|       | password_hash | VARCHAR(255)     |                  | Password (if using traditional login) |
|       | full_name     | VARCHAR(100)     | NOT NULL         | Full name                             |

| Table                 | Column      | Data Type        | Constraints              | Description                                                                                              |
|-----------------------|-------------|------------------|--------------------------|----------------------------------------------------------------------------------------------------------|
|                       | role        | VARCHAR (ENUM)   | NOT NULL                 | Role discriminator: STUDENT, LECTURER, ADMIN (determines which child table to JOIN)                      |
|                       | created_at  | TIMESTAMP        | DEFAULT NOW()            | Account creation timestamp                                                                               |
| <b>STUDENTS</b>       | user_id     | UNIQUEIDENTIFIER | PK, FK -> USERS(id)      | 1-to-1 with USERS. Reserved for future student-specific attributes (e.g., student_code, enrollment_year) |
| <b>LECTURERS</b>      | user_id     | UNIQUEIDENTIFIER | PK, FK -> USERS(id)      | 1-to-1 with USERS. Reserved for future attributes (e.g., department, academic_title)                     |
| <b>ADMINS</b>         | user_id     | UNIQUEIDENTIFIER | PK, FK -> USERS(id)      | 1-to-1 with USERS. Reserved for future attributes (e.g., permission_level)                               |
| <b>CLASSES</b>        | id          | UNIQUEIDENTIFIER | PK                       | Primary key, class identifier                                                                            |
|                       | class_code  | VARCHAR(50)      | NOT NULL                 | Class code (e.g., SE1801)                                                                                |
|                       | lecturer_id | UNIQUEIDENTIFIER | FK -> LECTURERS(user_id) | Lecturer in charge of the class                                                                          |
| <b>CLASS_STUDENTS</b> | class_id    | UNIQUEIDENTIFIER | FK -> CLASSES(id)        | Foreign key to class                                                                                     |

| Table              | Column           | Data Type        | Constraints             | Description                                         |
|--------------------|------------------|------------------|-------------------------|-----------------------------------------------------|
|                    | student_id       | UNIQUEIDENTIFIER | FK -> STUDENTS(user_id) | Foreign key to student                              |
| <b>ASSIGNMENTS</b> | id               | UNIQUEIDENTIFIER | PK                      | Primary key, assignment identifier                  |
|                    | class_id         | UNIQUEIDENTIFIER | FK -> CLASSES(id)       | Foreign key to class                                |
|                    | title            | VARCHAR(255)     | NOT NULL                | Assignment title                                    |
|                    | description      | TEXT             |                         | Assignment description                              |
|                    | language         | VARCHAR(20)      | NOT NULL                | Programming language for Sandbox (Java, Python, C#) |
|                    | deadline         | TIMESTAMP        | NOT NULL                | Submission deadline                                 |
|                    | grading_criteria | NVARCHAR(MAX)    |                         | AI-generated grading criteria                       |
| <b>TEST_CASES</b>  | id               | UNIQUEIDENTIFIER | PK                      | Primary key, test case identifier                   |
|                    | assignment_id    | UNIQUEIDENTIFIER | FK -> ASSIGNMENTS(id)   | Belongs to which assignment                         |
|                    | input_data       | TEXT             |                         | Input (StdIn)                                       |
|                    | expected_output  | TEXT             |                         | Expected output (StdOut)                            |
|                    | score_weight     | INT              | NOT NULL                | Score weight                                        |
| <b>SUBMISSIONS</b> | id               | UNIQUEIDENTIFIER | PK                      | Primary key, submission identifier                  |
|                    | assignment_id    | UNIQUEIDENTIFIER | FK -> ASSIGNMENTS(id)   | Belongs to which assignment                         |
|                    | student_id       | UNIQUEIDENTIFIER | FK -> STUDENTS(user_id) | Submitter                                           |
|                    | file_url         | VARCHAR(255)     | NOT NULL                | File link .zip (S3/Local)                           |

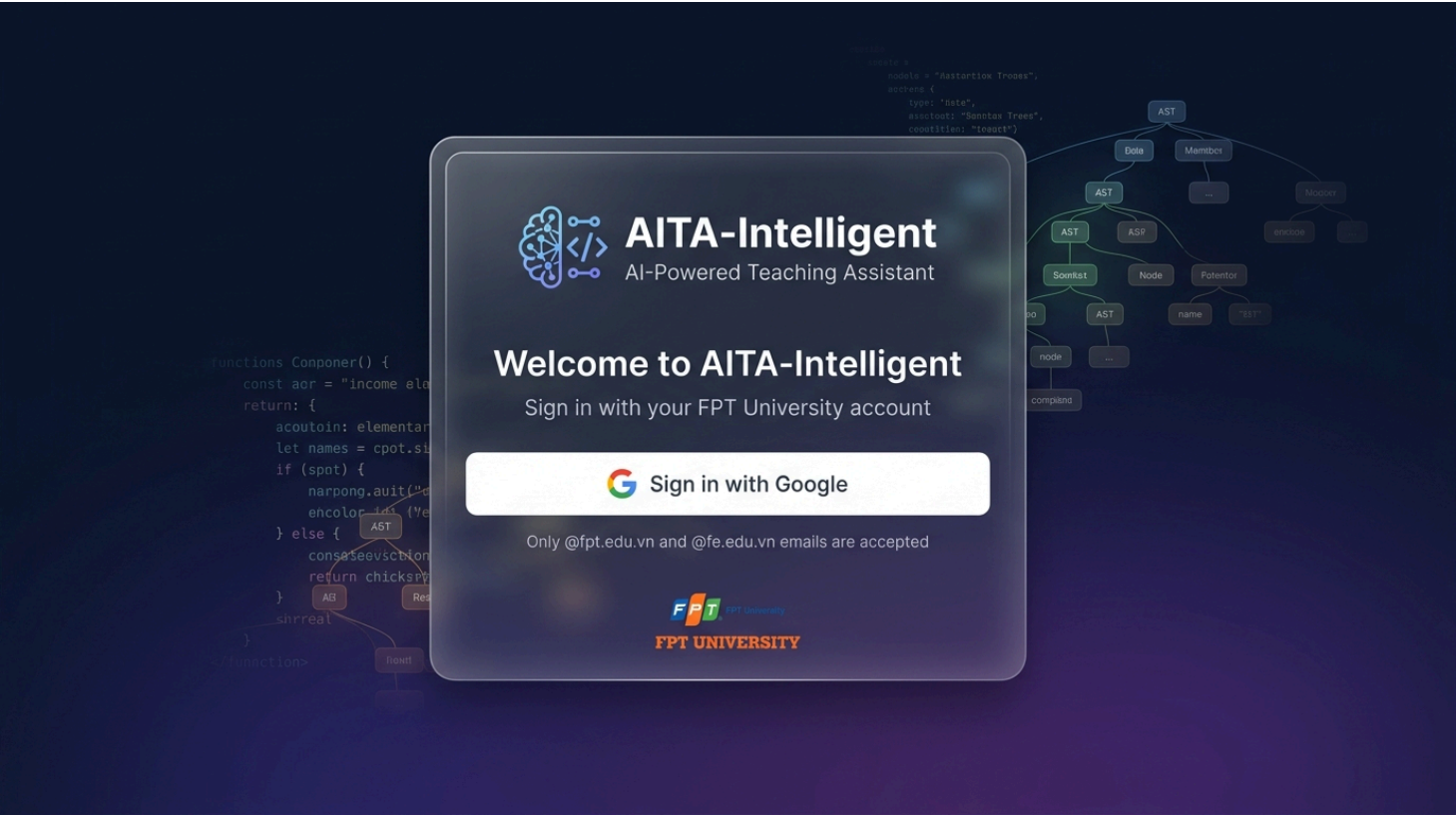
| Table                     | Column             | Data Type        | Constraints           | Description                                                        |
|---------------------------|--------------------|------------------|-----------------------|--------------------------------------------------------------------|
|                           | file_hash          | VARCHAR(64)      |                       | SHA-256 for integrity verification                                 |
|                           | status             | VARCHAR (ENUM)   | NOT NULL              | Status (PENDING, QUEUED...) (Mapped as string due to MSSQL limits) |
|                           | total_score        | FLOAT            |                       | Total score                                                        |
|                           | ai_feedback        | TEXT             |                       | AI Clean Code feedback                                             |
|                           | submitted_at       | TIMESTAMP        | DEFAULT NOW()         | Date and time of submission                                        |
| <b>AST_FINGERPRINTS</b>   | id                 | UNIQUEIDENTIFIER | PK                    | Primary key, AST fingerprint identifier                            |
|                           | submission_id      | UNIQUEIDENTIFIER | FK -> SUBMISSIONS(id) | Fingerprint of which submission                                    |
|                           | fingerprint_data   | NVARCHAR(MAX)    | NOT NULL              | Winnowing k-gram hash array                                        |
| <b>PLAGIARISM_MATCHES</b> | id                 | UNIQUEIDENTIFIER | PK                    | Primary key, match identifier                                      |
|                           | submission_1_id    | UNIQUEIDENTIFIER | FK -> SUBMISSIONS(id) | First submission                                                   |
|                           | submission_2_id    | UNIQUEIDENTIFIER | FK -> SUBMISSIONS(id) | Second submission                                                  |
|                           | similarity_percent | FLOAT            | NOT NULL              | Percentage of similarity (0-100)                                   |
|                           | matched_fragments  | NVARCHAR(MAX)    |                       | Array of matching line numbers/blocks                              |
| <b>APPEALS</b>            | id                 | UNIQUEIDENTIFIER | PK                    | Primary key, appeal identifier                                     |
|                           | submission_id      | UNIQUEIDENTIFIER | FK -> SUBMISSIONS(id) | Appeal for which submission                                        |

| Table                   | Column            | Data Type        | Constraints                        | Description                                                                               |
|-------------------------|-------------------|------------------|------------------------------------|-------------------------------------------------------------------------------------------|
|                         | student_id        | UNIQUEIDENTIFIER | FK -> STUDENTS(user_id)            | Foreign key to student who created the appeal                                             |
|                         | resolved_by       | UNIQUEIDENTIFIER | FK -> LECTURERS(user_id), NULLABLE | Lecturer who resolved the appeal (NULL while PENDING)                                     |
|                         | reason            | TEXT             | NOT NULL                           | Student's appeal reason                                                                   |
|                         | status            | ENUM             | DEFAULT 'PENDING'                  | Status (PENDING, APPROVED, REJECTED)                                                      |
|                         | lecturer_response | TEXT             |                                    | Lecturer's response                                                                       |
|                         | created_at        | TIMESTAMP        | DEFAULT NOW()                      | Timestamp of creation                                                                     |
|                         | resolved_at       | TIMESTAMP        |                                    | Timestamp of resolution                                                                   |
| <b>SCORE_AUDIT_LOGS</b> | id                | UNIQUEIDENTIFIER | PK                                 | Primary key, log identifier                                                               |
|                         | submission_id     | UNIQUEIDENTIFIER | FK -> SUBMISSIONS(id)              | Affected submission                                                                       |
|                         | editor_id         | UNIQUEIDENTIFIER | FK -> USERS(id)                    | Who made the edit (Lecturer or Admin — intentionally references USERS, not a child table) |
|                         | old_score         | FLOAT            |                                    | Previous score                                                                            |
|                         | new_score         | FLOAT            |                                    | Updated score                                                                             |
|                         | reason            | TEXT             | NOT NULL                           | Reason for score change                                                                   |
|                         | created_at        | TIMESTAMP        | DEFAULT NOW()                      | Timestamp of edit                                                                         |

## 6.7. UI Wireframes & Screen Flow

This section describes the key screens of the AITA-Intelligent system, their layout, components, and the navigation flow between them.

### 6.7.1. Screen 1 — Login Page (Google SSO)



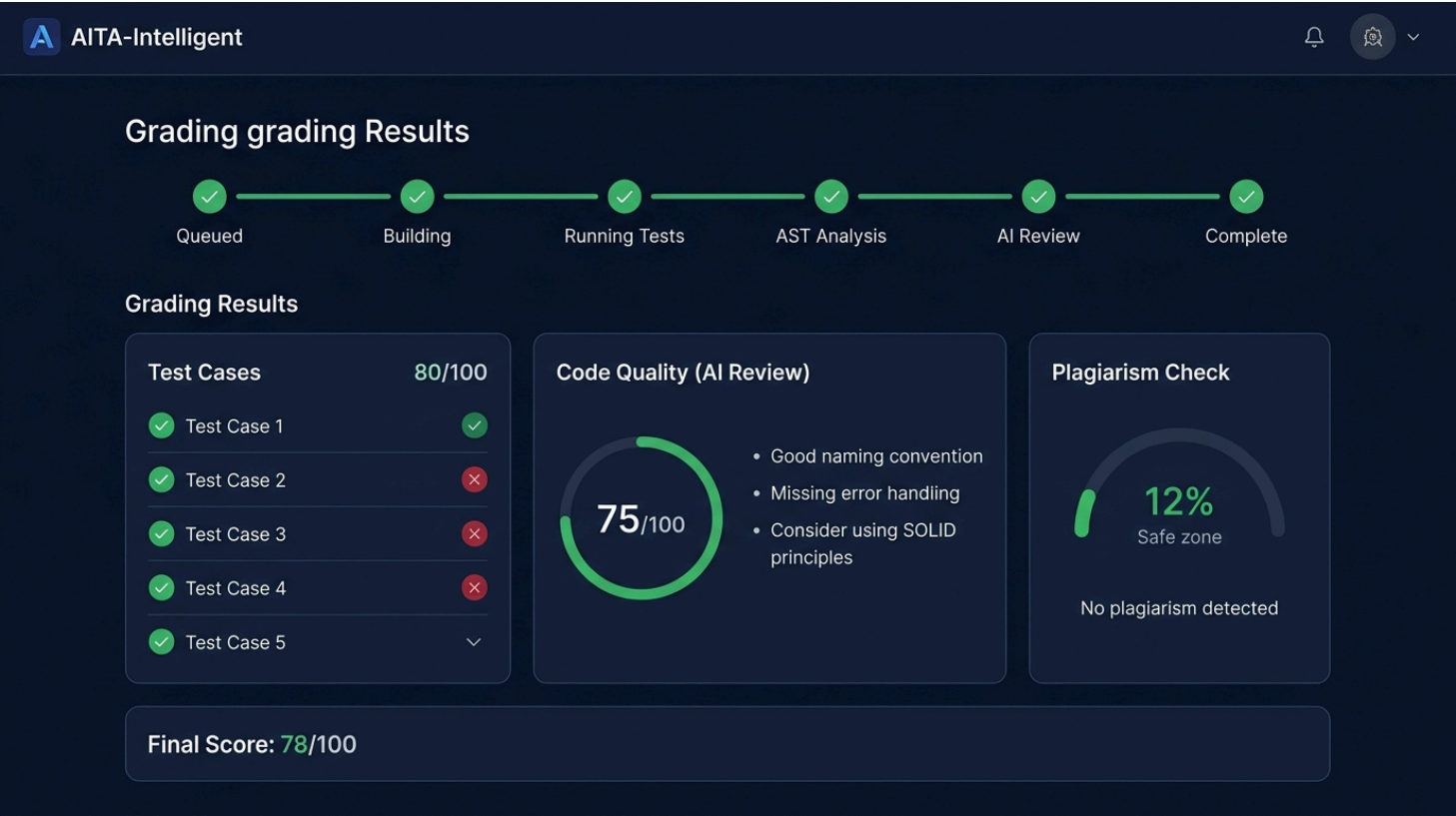
| Attribute  | Description                                                                                                                     |
|------------|---------------------------------------------------------------------------------------------------------------------------------|
| URL        | /login                                                                                                                          |
| Layout     | Centered card on gradient background                                                                                            |
| Components | FPT University logo, Application title "AITA-Intelligent", "Sign in with Google" button (primary CTA), Footer with version info |
| Actions    | Click "Sign in with Google" → Google OAuth 2.0 consent screen → Redirect to /dashboard on success                               |
| Validation | Only @fpt.edu.vn and @fe.edu.vn email domains accepted. Others shown error toast.                                               |

### 6.7.2. Screen 2 — Student Dashboard

| Attribute  | Description                                                                                                         |
|------------|---------------------------------------------------------------------------------------------------------------------|
| URL        | /dashboard (role: Student)                                                                                          |
| Layout     | Top navbar (avatar, notifications, logout) + Main content area with assignment cards                                |
| Components | - <b>Assignment Cards:</b> Title, Class code, Deadline countdown, Status badge (Not Submitted / Submitted / Graded) |

| Attribute | Description                                                                                                                                                                                      |
|-----------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|           | <ul style="list-style-type: none"><li>- <b>Sort/Filter:</b> By deadline (nearest first), by status</li><li>- <b>Quick Stats bar:</b> Total assignments, Submitted count, Average score</li></ul> |
| Actions   | Click card → Navigate to /assignments/:id/submit<br>Click graded card → Navigate to /submissions/:id/results                                                                                     |
| Real-time | WebSocket listens for grading completion events → Status badge auto-updates without page reload                                                                                                  |

6.7.3. Screen 3 — Submission & Grading Results



| Attribute                        | Description                                                                                                                                |
|----------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------|
| URL                              | /submissions/:id/results                                                                                                                   |
| Layout                           | Progress stepper (top) + 3 result cards (horizontal row) + Final score bar (bottom)                                                        |
| Components                       |                                                                                                                                            |
| Progress Stepper                 | 6 steps: Queued → Building → Running Tests → AST Analysis → AI Review → Complete. Active step: yellow spinner. Completed: green checkmark. |
| Card 1: Test Cases               | Title + score (e.g., 80/100). List of test cases with ✓ (passed) / ✗ (failed). Expandable: Input, Expected Output, Actual Output.          |
| Card 2: Code Quality (AI Review) | Circular gauge (0-100). AI feedback list: "✓ Good naming convention", "⚠ Missing error handling", "💡 Consider SOLID principles".           |
| Card 3: Plagiarism Check         | Semicircle gauge with color zones: 0-30% green (Safe), 30-60% yellow (Warning), 60-100% red (Danger). Match details if flagged.            |

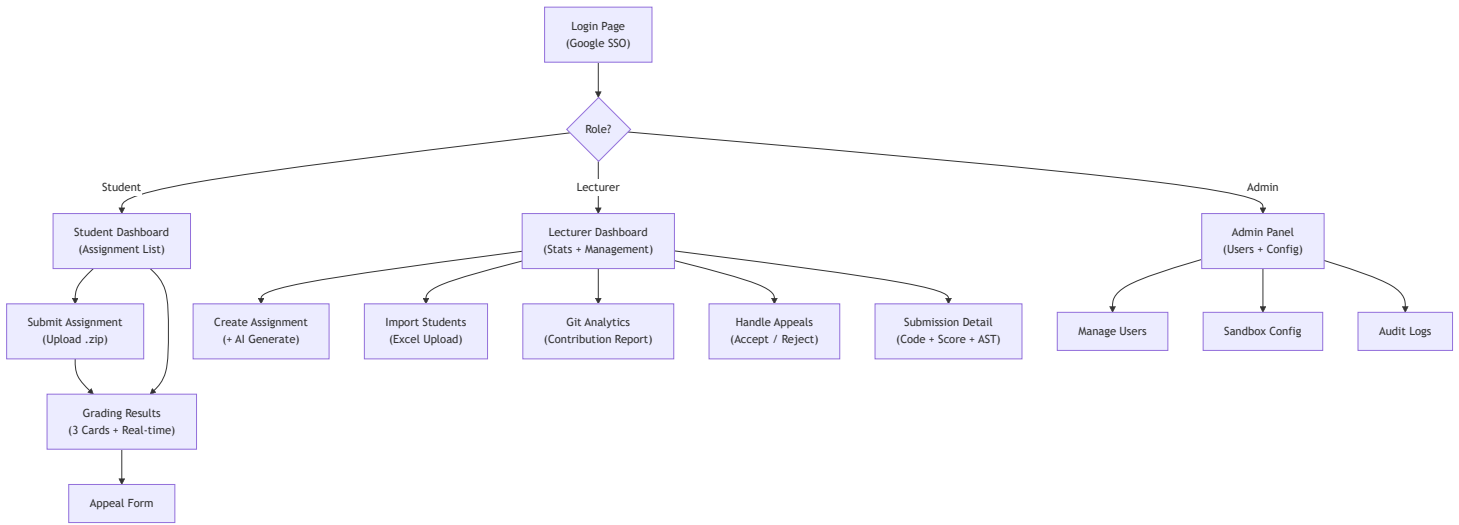
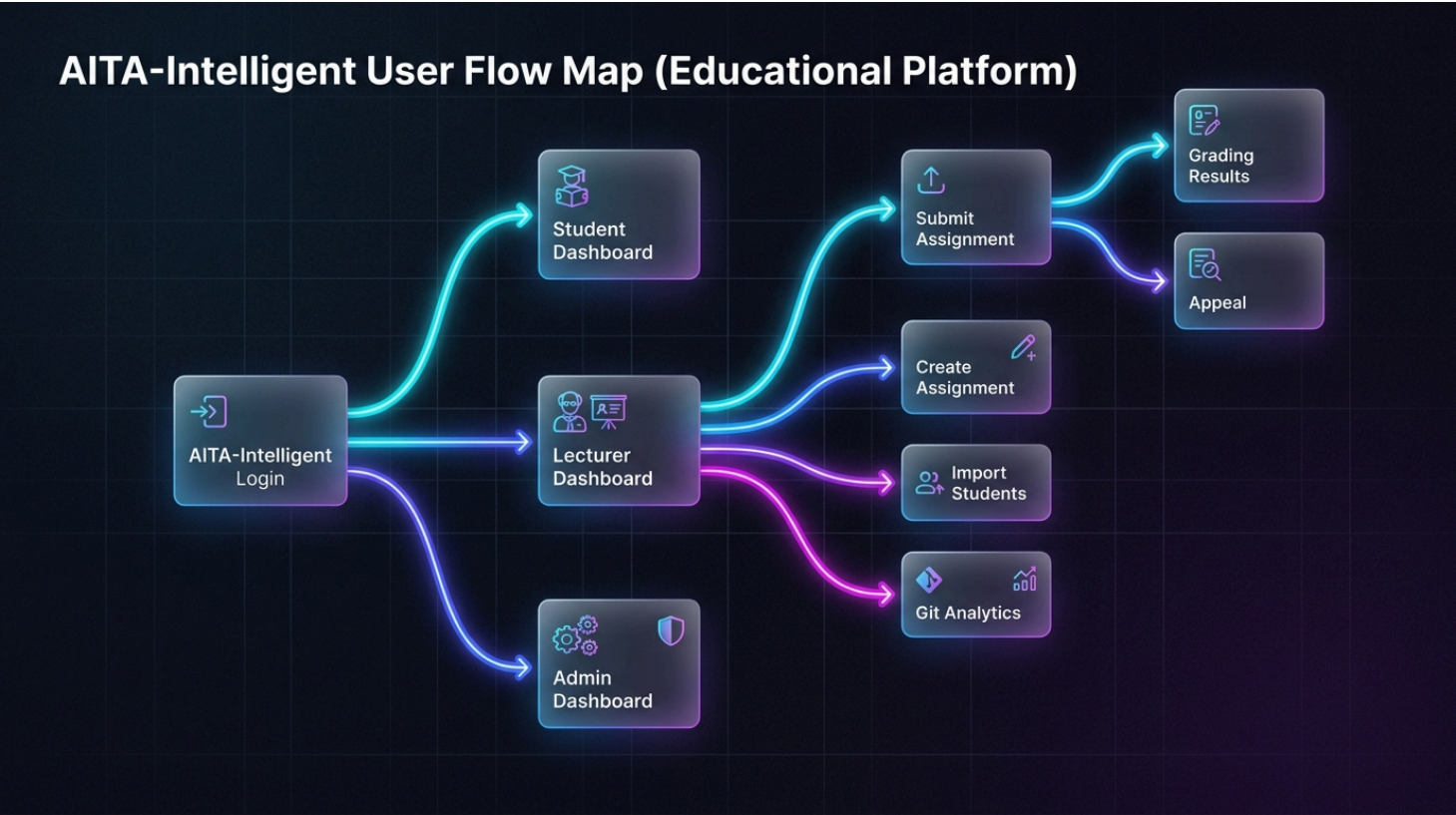
| Attribute       | Description                                                                                                                                                                             |
|-----------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Final Score Bar | "Final Score: XX/100" (large font). If status is SCORE_WITHHELD, display "Score pending lecturer review due to suspected plagiarism". "Appeal this result" button if student disagrees. |
| Real-time       | WebSocket connection pushes step-by-step progress updates. All 3 cards render simultaneously on "Complete".                                                                             |

6.7.4. Screen 4 — Lecturer Dashboard



| Attribute                 | Description                                                                                                                            |
|---------------------------|----------------------------------------------------------------------------------------------------------------------------------------|
| URL                       | /dashboard (role: Lecturer)                                                                                                            |
| Layout                    | Sidebar navigation + Main content with stats cards and data tables                                                                     |
| Components                |                                                                                                                                        |
| Sidebar                   | Class list, Create Assignment, Import Students, Git Analytics, Appeals, Settings                                                       |
| Stats Cards (top row)     | Total Students, Submissions Today, Average Score, Pending Appeals                                                                      |
| Submission Rate Chart     | Pie chart: Submitted vs Not Submitted vs Late                                                                                          |
| Score Distribution Chart  | Bar chart: score ranges (0-20, 20-40, 40-60, 60-80, 80-100)                                                                            |
| Plagiarism Suspects Table | Top 10 submission pairs with highest AST similarity %, with "View Detail" link                                                         |
| Actions                   | Click "Create Assignment" → /assignments/new<br>Click "Import Students" → Upload modal<br>Click student row → Detailed submission view |

6.7.5. Screen Flow Diagram



7. Non-Functional Requirements

7.1. Security

| ID     | Requirement                                                             | Measurement Criteria                                                                                              |
|--------|-------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------|
| NFR-01 | Code Execution Engine (Judge0/Piston) must completely isolate resources | RAM ≤ 512MB, limited CPU shares, <b>external network 100% disabled</b> (provided by Judge0/Piston out-of-the-box) |

| ID     | Requirement                                                            | Measurement Criteria                                                                                                                          |
|--------|------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------|
| NFR-02 | Execution sandbox must defend against common malware                   | Block: fork bomb, symlink escape, /proc mount, disk exhaustion (Judge0/Piston handles natively)                                               |
| NFR-03 | JWT authentication must be stored in HttpOnly Cookie                   | Cookie inaccessible from JavaScript (XSS prevention)                                                                                          |
| NFR-04 | Google SSO must only accept FPT email domains                          | Whitelist: @fpt.edu.vn , @fe.edu.vn                                                                                                           |
| NFR-05 | API keys (OpenAI, Gemini) must not be hard-coded                       | Store in environment variables (.env), never commit to Git                                                                                    |
| NFR-18 | <b>(P2P Defense)</b> System must log every sandbox execution in detail | Log includes: submission_id , start_time , end_time , exit_code , resource_usage (RAM/CPU peak), security_alert . Retain for minimum 30 days. |

## 7.2. Performance

| ID     | Requirement                            | Measurement Criteria                                                                                                                                                |
|--------|----------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| NFR-06 | Average API response time              | ≤ 200ms for standard APIs (CRUD)                                                                                                                                    |
| NFR-07 | End-to-end grading time per submission | ≤ 90 seconds (includes: Code Execution + AST + AI). <i>Note: The 30s limit applies to code execution in Sandbox; 90s is the total time for the entire pipeline.</i> |
| NFR-08 | System must handle concurrent load     | ≥ 100 requests/minute (stress test with k6 or JMeter)                                                                                                               |
| NFR-09 | Grading results must be cached         | Redis cache for result APIs, TTL = 5 minutes, response ≤ 100ms                                                                                                      |

## 7.3. Scalability

| ID     | Requirement                                          | Measurement Criteria                                                        |
|--------|------------------------------------------------------|-----------------------------------------------------------------------------|
| NFR-10 | Redis Queue must support batch grading               | Process ≥ 50 concurrent submissions without blocking API gateway            |
| NFR-11 | System must support adding new programming languages | Configure new language ID in Judge0 API + add ANTLR grammar for AST support |

## 7.4. Reliability

| ID     | Requirement                                 | Measurement Criteria                                |
|--------|---------------------------------------------|-----------------------------------------------------|
| NFR-12 | Bulk student import must use DB Transaction | Import all successfully OR rollback entirely (ACID) |

| ID     | Requirement                                        | Measurement Criteria                                                                 |
|--------|----------------------------------------------------|--------------------------------------------------------------------------------------|
| NFR-13 | Redis Queue must have retry mechanism              | Max 3 retries with exponential backoff                                               |
| NFR-14 | Sandbox resources must be cleaned after completion | Judge0 API handles process isolation and cleanup; system verifies via status polling |

## 7.5. Usability

| ID     | Requirement                                  | Measurement Criteria                                                    |
|--------|----------------------------------------------|-------------------------------------------------------------------------|
| NFR-15 | Responsive interface on both Web and Mobile  | Support: Desktop (≥ 1024px), Tablet (≥ 768px), Mobile (≥ 375px)         |
| NFR-16 | Real-time result updates without page reload | Using WebSocket ( <a href="#">Socket.IO</a> ) with API polling fallback |
| NFR-17 | Compilation error explanations in Vietnamese | AI returns explanations in Vietnamese natural language                  |

# 8. System Constraints

## 8.1. Technology Constraints

- **Frontend Web:** React 18+ with Vite, TailwindCSS
- **Backend:** Node.js 20+ with TypeScript, Prisma ORM
- **AST Engine:** Python 3.11+ with FastAPI, ANTLR4 ( `antlr4-python3-runtime` for Java grammar)
- **Database:** Microsoft SQL Server 2019+
- **Message Queue:** Redis 7+ with BullMQ
- **Code Execution:** Judge0 / Piston (self-hosted or API)
- **CI/CD:** GitHub Actions
- **Cloud Deploy:** Azure / AWS / Vercel (team's choice)

## 8.2. MVP Scope Limitations

The following features are scoped as **simplified/secondary** for the 10-week MVP. Full implementations are planned for future iterations.

| Feature                             | MVP Scope                                                           | Future Scope                                                            |
|-------------------------------------|---------------------------------------------------------------------|-------------------------------------------------------------------------|
| AST Plagiarism Detection            | Java only (ANTLR parser)                                            | Add Python ( <code>ast</code> module), C# (Roslyn)                      |
| GenAI Assignment Generation (US-08) | Basic draft generation (assignment description + sample test cases) | Full rubric auto-generation, multi-round refinement                     |
| Git Analytics (US-11)               | Basic metrics: commits count, LOC per member                        | PR analysis, contribution timeline charts, automated free-rider scoring |

| Feature               | MVP Scope                                 | Future Scope                                                |
|-----------------------|-------------------------------------------|-------------------------------------------------------------|
| Code Execution Engine | Use Judge0/Piston API (pre-built sandbox) | Custom Docker containers with fine-grained resource control |
| Mobile App            | Responsive web (PWA-ready)                | Native React Native / Expo app                              |

### 8.3. Risk Assessment

| #    | Risk                                                                                                                     | Impact                                                       | Mitigation                                                                                                                                                                    |
|------|--------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| R-01 | <b>Resubmission Policy</b> — No clear rule on how many times a student can resubmit before deadline                      | Students may flood the queue with unlimited resubmissions    | Limit to max 5 submissions per assignment per student. Show remaining attempts on UI.                                                                                         |
| R-02 | <b>API Rate Limiting</b> — OpenAI/Gemini API may throttle or reject requests during peak hours                           | AI Code Review step fails, resulting in incomplete grading   | Implement queue-based retry with exponential backoff. Fallback: skip AI review and mark as "AI Review Pending".                                                               |
| R-03 | <b>Audit Trail for Manual Edits</b> — Lecturer manually overrides a score but no log is kept                             | Disputes during P2P Defense cannot be resolved with evidence | Log every manual score edit: <code>editor_id</code> , <code>old_score</code> , <code>new_score</code> , <code>reason</code> , <code>timestamp</code> . Immutable audit table. |
| R-04 | <b>False-Positive Plagiarism</b> — AST + Winnowing flags legitimate code as plagiarism (e.g., boilerplate, starter code) | Students unfairly penalized, appeals increase                | Allow lecturer to whitelist specific code patterns/files. Display similarity breakdown (which functions matched). Student can appeal with explanation.                        |
| R-05 | <b>Data Retention Policy</b> — No defined policy for how long submission files, logs, and personal data are stored       | GDPR/PDPA compliance risk, storage costs grow unbounded      | Define retention: Submission files = 1 semester, Sandbox logs = 30 days (NFR-18), Personal data = until account deletion. Auto-purge scripts run monthly.                     |

### 8.4. Business Constraints

- The system only serves students and lecturers belonging to FPT University (verified by email domain).
- Each submission is limited to a maximum of 10MB (.zip file).
- Maximum code execution time is 30 seconds per submission (enforced by Judge0 API timeout).
- OpenAI/Gemini API key uses a single key with exponential backoff retry.

### 8.5. Project Constraints

- Development timeline: 10 weeks.
- Development team: 4-6 students.
- Code must pass linting (ESLint/Prettier for TypeScript, Flake8 for Python).
- Unit test coverage  $\geq 80\%$  on core modules (scoring, validation, matching logic - excluding live Judge0/AI integration paths).

